Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Математические основы интеллектуальных систем»

За 1 семестр

Тема: «Нахождение компонент связности неориентированного графа»

Вариант 4

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.

Проверил:
Козинский А.А.

Брест 2026

Цель работы: Изучить компоненты связанности, методы обходов графов, алгоритм нахождения компонент связности графа.

Ход работы:

№	Кол. вершин	Кол. ребер	Задание графа
4.	8	6	(1,6),(1,8),2,5),(3,6),(3,8),(4,5)

Задание 1:

Построить матрицу смежности и инцидентности для заданного графа. Изобразить граф.

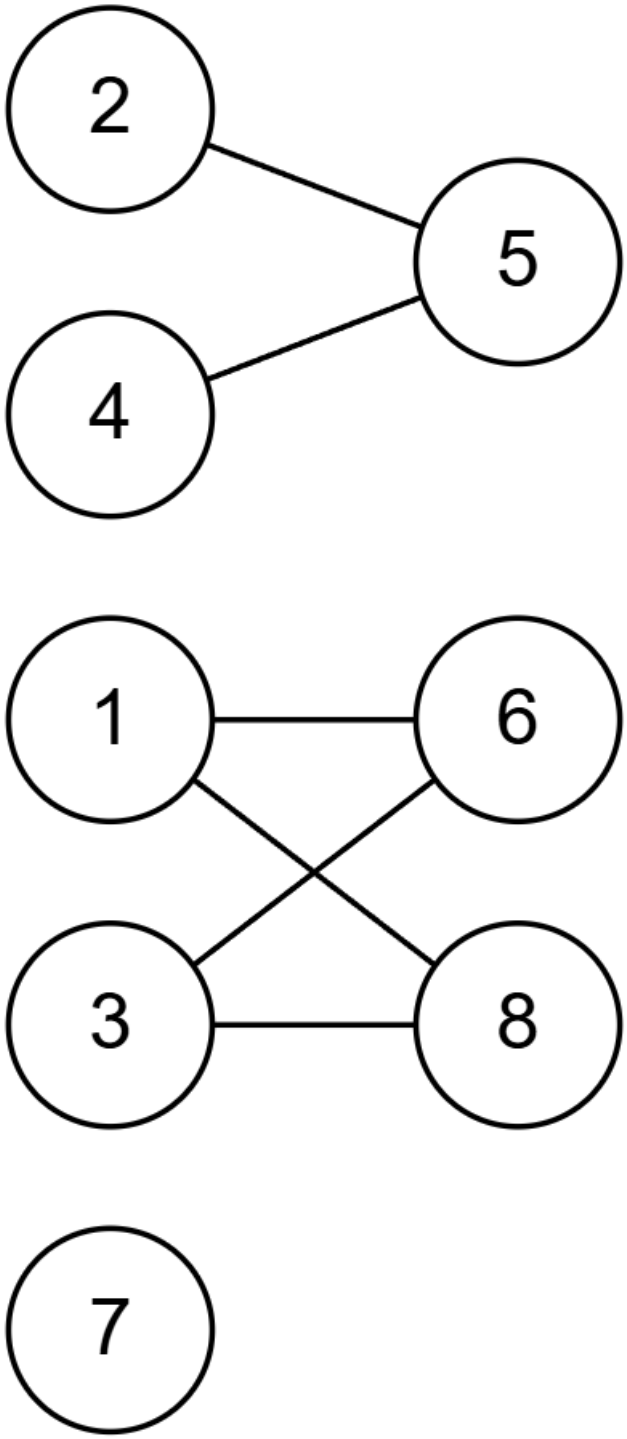
Матрица смежности:

	1	2	3	4	5	6	7	8
1	0	0	0	0	0	1	0	1
2	0	0	0	0	1	0	0	0
3	0	0	0	0	0	1	0	1
4	0	0	0	0	1	0	0	0
5	0	1	0	1	0	0	0	0
6	1	0	1	0	0	0	0	0
7	0	0	0	0	0	0	0	0
8	1	0	1	0	0	0	0	0

Матрица инцидентности:

	1	2	3	4	5	6
1	1	1	0	0	0	0
2	0	0	1	0	0	0
3	0	0	0	1	1	0
4	0	0	0	0	0	1
5	0	0	1	0	0	1
6	1	0	0	1	0	0
7	0	0	0	0	0	0
8	0	1	0	0	1	0

Граф(Визуально):



Задание 2: Используя поиск в глубину и поиск в ширину написать программу, определяющую число компонент связности графа. Методы представляются в виде функций.

Код программы:

```
#include <stdio.h>


#define MAX_V 100
```

```
#define MAX_E 100

int adjacency[MAX_V][MAX_V];
int incidence[MAX_V][MAX_E];
int edge_u[MAX_E];
int edge_v[MAX_E];
int visited[MAX_V];
int components[MAX_V][MAX_V];
int comp_sizes[MAX_V];
int queue[MAX_V];
int stack[MAX_V];

int read_graph(const char *filename, int *n_out, int *m_out)
{
    FILE *f = fopen(filename, "r");
    if (!f)
        return 0;

    int n = 0;
    int m = 0;

    if (fscanf(f, "%d %d", &n, &m) != 2)
    {
        fclose(f);
        return 0;
    }

    if (n < 1 || n > MAX_V || m < 0 || m > MAX_E)
    {
        fclose(f);
        return 0;
    }
```

```
    int edge_count = 0;

    for (int i = 0; i < m; i++)
    {
        int u = 0;
        int v = 0;

        if (fscanf(f, "%d %d", &u, &v) != 2)
        {
            break;
        }

        if (u >= 1 && u <= n && v >= 1 && v <= n)
        {
            edge_u[edge_count] = u;
            edge_v[edge_count] = v;
            edge_count++;
        }
    }

    fclose(f);
    *n_out = n;
    *m_out = edge_count;
    return 1;
}

void build_adjacency(int n, int m)
{
    for (int i = 0; i < MAX_V; i++)
    {
        for (int j = 0; j < MAX_V; j++)
```

```
        {
            adjacency[i][j] = 0;
        }
    }

    for (int i = 0; i < m; i++)
    {
        int u = edge_u[i] - 1;
        int v = edge_v[i] - 1;

        adjacency[u][v] = 1;
        adjacency[v][u] = 1;
    }
}

void build_incidence(int n, int m)
{
    for (int i = 0; i < MAX_V; i++)
    {
        for (int j = 0; j < MAX_E; j++)
        {
            incidence[i][j] = 0;
        }
    }

    for (int i = 0; i < m; i++)
    {
        int u = edge_u[i] - 1;
        int v = edge_v[i] - 1;

        if (u == v)
        {
```

```
            incidence[u][i] = 2;
        }
        else
        {
            incidence[u][i] = 1;
            incidence[v][i] = 1;
        }
    }
}

void sort_component(int *arr, int len)
{
    for (int i = 0; i < len; i++)
    {
        for (int j = i + 1; j < len; j++)
        {
            if (arr[i] > arr[j])
            {
                int t = arr[i];
                arr[i] = arr[j];
                arr[j] = t;
            }
        }
    }
}

int bfs_components(int n)
{
    for (int i = 0; i < n; i++)
    {
        visited[i] = 0;
    }
```

```
int count = 0;

for (int start = 0; start < n; start++)
{
    if (!visited[start])
    {
        int size = 0;
        int head = 0;
        int tail = 0;

        queue[tail++] = start;
        visited[start] = 1;

        while (head < tail)
        {
            int v = queue[head++];
            components[count][size++] = v;

            for (int to = 0; to < n; to++)
            {
                if (adjacency[v][to] && !visited[to])
                {
                    visited[to] = 1;
                    queue[tail++] = to;
                }
            }
        }

        sort_component(components[count], size);
        comp_sizes[count] = size;
        count++;
```

```
        }
    }

    return count;
}

int dfs_components(int n)
{
    for (int i = 0; i < n; i++)
    {
        visited[i] = 0;
    }

    int count = 0;

    for (int start = 0; start < n; start++)
    {
        if (!visited[start])
        {
            int size = 0;
            int top = 0;

            stack[top++] = start;
            visited[start] = 1;

            while (top > 0)
            {
                int v = stack[--top];
                components[count][size++] = v;

                for (int to = 0; to < n; to++)
                {
```

```
                    if (adjacency[v][to] && !visited[to])
                    {
                        visited[to] = 1;
                        stack[top++] = to;
                    }
                }
            }

            sort_component(components[count], size);
            comp_sizes[count] = size;
            count++;
        }
    }

    return count;
}

void print_edges(int m)
{
    printf("Edges:");

    for (int i = 0; i < m; i++)
    {
        printf(" (%d,%d)", edge_u[i], edge_v[i]);
    }

    printf("\n");
}

void print_adjacency(int n)
{
    for (int i = 0; i < n; i++)
```

```
    {
        for (int j = 0; j < n; j++)
        {
            printf("%d ", adjacency[i][j]);
        }
        printf("\n");
    }
}

void print_incidence(int n, int m)
{
    if (m == 0)
    {
        printf("No edges\n");
        return;
    }

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            printf("%d ", incidence[i][j]);
        }
        printf("\n");
    }
}

void print_components(int count)
{
    for (int i = 0; i < count; i++)
    {
        printf("Component %d:", i + 1);
```

```
        for (int j = 0; j < comp_sizes[i]; j++)
        {
            printf(" %d", components[i][j] + 1);
        }

        printf("\n");
    }
}

int main(int argc, char *argv[])
{
    const char *filename = argc > 1 ? argv[1] : "graph1.txt";

    int n = 0;
    int m = 0;

    if (!read_graph(filename, &n, &m))
    {
        printf("Cannot read file: %s\n", filename);
        return 1;
    }

    build_adjacency(n, m);
    build_incidence(n, m);

    printf("File: %s\n", filename);
    printf("Vertices: %d, Edges: %d\n", n, m);
    print_edges(m);

    printf("Adjacency matrix:\n");
    print_adjacency(n);
```

```
    printf("Incidence matrix:\n");
    print_incidence(n, m);

    int count = bfs_components(n);
    printf("BFS components: %d\n", count);
    print_components(count);

    count = dfs_components(n);
    printf("DFS components: %d\n", count);
    print_components(count);

    return 0;
}
```

Результат:

Тест 1:

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph1.txt
File: .\graph1.txt
Vertices: 9, Edges: 10
Edges: (1,2) (1,4) (1,6) (2,3) (2,5) (3,5) (4,5) (4,6) (7,8) (7,9)
Adjacency matrix:
0 1 0 1 0 1 0 0 0
1 0 1 0 1 0 0 0 0
0 1 0 0 1 0 0 0 0
1 0 0 0 1 1 0 0 0
0 1 1 1 0 0 0 0 0
1 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 1 1
0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 1 0 0
Incidence matrix:
1 1 1 0 0 0 0 0 0 0
1 0 0 1 1 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0
0 1 0 0 0 0 1 1 0 0
0 0 0 0 1 1 1 0 0 0
0 0 1 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 1 1
0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 1
BFS components: 2
Component 1: 1 2 3 4 5 6
Component 2: 7 8 9
DFS components: 2
Component 1: 1 2 3 4 5 6
Component 2: 7 8 9
```

Тест 2:

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph2.txt
File: .\graph2.txt
Vertices: 6, Edges: 8
Edges: (1,2) (1,4) (1,5) (1,6) (2,3) (3,4) (4,5) (5,6)
Adjacency matrix:
0 1 0 1 1 1
1 0 1 0 0 0
0 1 0 1 0 0
1 0 1 0 1 0
1 0 0 1 0 1
1 0 0 0 1 0
Incidence matrix:
1 1 1 1 0 0 0 0
1 0 0 0 1 0 0 0
0 0 0 0 1 1 0 0
0 1 0 0 0 1 1 0
0 0 1 0 0 0 1 1
0 0 0 1 0 0 0 1
BFS components: 1
Component 1: 1 2 3 4 5 6
DFS components: 1
Component 1: 1 2 3 4 5 6
```

Тест 3:

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph3.txt
File: .\graph3.txt
Vertices: 8, Edges: 8
Edges: (1,2) (1,3) (2,3) (3,4) (3,5) (4,5) (5,6) (7,8)
Adjacency matrix:
0 1 1 0 0 0 0 0
1 0 1 0 0 0 0 0
1 1 0 1 1 0 0 0
0 0 1 0 1 0 0 0
0 0 1 1 0 1 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0
Incidence matrix:
1 1 0 0 0 0 0 0
1 0 1 0 0 0 0 0
0 1 1 1 1 0 0 0
0 0 0 1 0 1 0 0
0 0 0 0 1 1 1 0
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
BFS components: 2
Component 1: 1 2 3 4 5 6
Component 2: 7 8
DFS components: 2
Component 1: 1 2 3 4 5 6
Component 2: 7 8
```

Тест 4(Мой вариант):

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph4.txt
File: .\graph4.txt
Vertices: 8, Edges: 6
Edges: (1,6) (1,8) (2,5) (3,6) (3,8) (4,5)
Adjacency matrix:
0 0 0 0 0 1 0 1
0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 1
0 0 0 0 1 0 0 0
0 1 0 1 0 0 0 0
1 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0
Incidence matrix:
1 1 0 0 0 0
0 0 1 0 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 1 0 0 1
1 0 0 1 0 0
0 0 0 0 0 0
0 1 0 0 1 0
BFS components: 3
Component 1: 1 3 6 8
Component 2: 2 4 5
Component 3: 7
DFS components: 3
Component 1: 1 3 6 8
Component 2: 2 4 5
Component 3: 7
```

Тест 5:

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph5.txt
File: .\graph5.txt
Vertices: 1, Edges: 0
Edges:
Adjacency matrix:
0
Incidence matrix:
No edges
BFS components: 1
Component 1: 1
DFS components: 1
Component 1: 1
```

Тест 6:

```
PS C:\metamorphosq\MOIS> .\lab1.exe .\graph6.txt
File: .\graph6.txt
Vertices: 25, Edges: 23
Edges: (1,2) (2,3) (3,4) (4,5) (5,6) (6,7) (7,8) (8,9) (9,10) (1,10) (11,12) (11,13) (11,14) (11,15) (11,16) (11,17) (11,18)
 (11,19) (11,20) (21,22) (22,23) (23,24) (21,24)
Adjacency matrix:
0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
Incidence matrix:
1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
BFS components: 4
Component 1: 1 2 3 4 5 6 7 8 9 10
Component 2: 11 12 13 14 15 16 17 18 19 20
Component 3: 21 22 23 24
Component 4: 25
DFS components: 4
Component 1: 1 2 3 4 5 6 7 8 9 10
Component 2: 11 12 13 14 15 16 17 18 19 20
Component 3: 21 22 23 24
Component 4: 25
```

Вывод: в ходе лабораторной работы я изучил компоненты связанности, методы обходов графов, алгоритм нахождения компонент связности графа.